

# 1 Descrierea algoritmului

Algoritmul poate fi despartit in 4 etape principale: citirea input-urilor transmise programului prin argumente in linia de comanda, citirea metadatelor din fisierele JSON (reading), prelucrarea acestor data si construirea structurilor de date relevante cerintei (autorii, sortarea articolelor pe categorii si limbi, etc.) (indexing), si scrierea rezultatelor acestei prelucrari in fisierele de output.

Distingem doua categorii de thread-uri: cel principal (main), pe care incepe executia programului, respectiv cele auxiliare (workers), create de thread-ul principal si care sunt in numar variabil, egal cu primul argument transmis prin linia de comanda.

Din cele 4 stagii ale algoritmului, prima si ultima etapa sunt executate secvential, pe thread-ul principal, in timp ce etapele de indexing si reading sunt executate in paralel, exclusiv pe thread-urile "workers". Fiecare thread "worker" trece secvential prin urmatoarele 6 stari: (1) asteptarea ca thread-ul principal sa permita operatia de citire a metadatelor, (2) citirea efectiva a metadatelor, (3) asteptarea ca toate thread-urile "workers" sa termine starea de "reading", (4) asteptarea ca thread-ul principal sa permita operatia de prelucrare a articolelor, (5) prelucrarea efectiva a articolelor, (6) asteptarea ca toate thread-urile "workers" sa termine starea de "indexing".

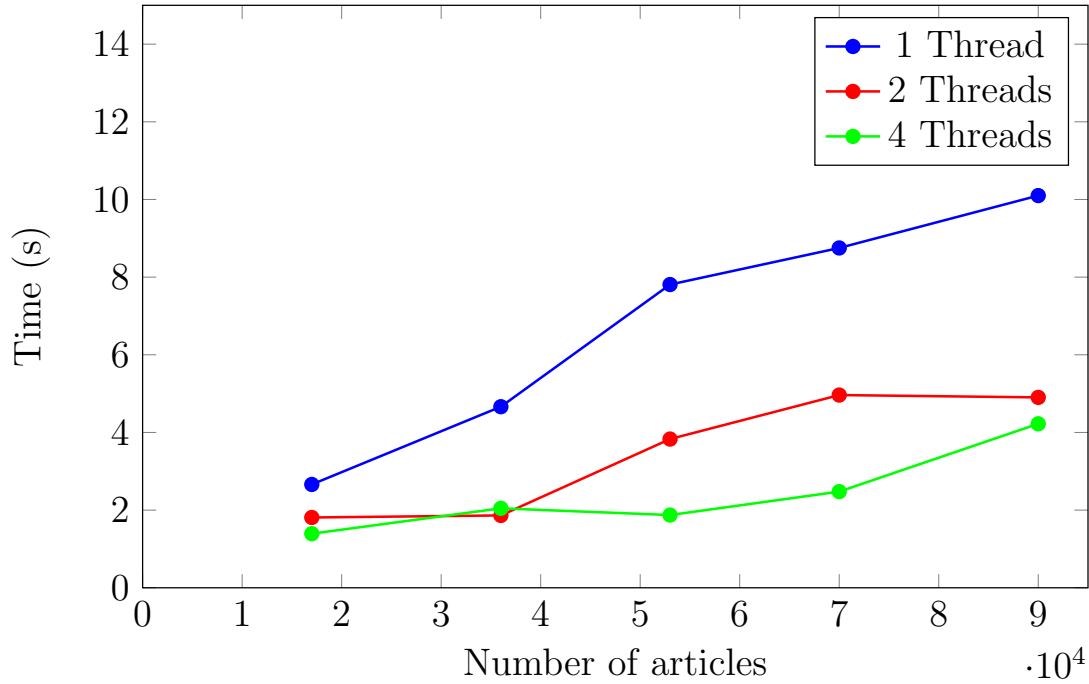
Sincronizarea intre aceste 6 stari se face exclusiv prin '**CountDownLatch**'-uri, gestionate de clasa '**Database**', ale carei functii ruleaza pe thread-ul principal. Pentru etapa (2), de "reading", thread-urile "workers" extrag filepath-urile fisierelor cu metadata dintr-un '**ConcurrentLinkedQueue**', care nu necesita sincronizare externa, iar rezultatele sunt stocate local de fiecare thread; etapa se termina atunci cand lista cu filepath-uri este goala, iar cand toate thread-urile "workers" trec in starea (3) thread-ul principal uneste toate rezultatele prin functia '**Database.bindReadingResultsFromWorkers**'. Aceeasi strategie de paralelizare este folosita si pentru etapa (5).

## 2 Rezultate brute

Tabelele de mai jos contin timpii bruti in care ruleaza algoritmul prezentat, in functie de marimea datelor de intrare (numarul de articole prelucrate), precum si numarul de thread-uri de tip "workers" alocate. Datele de intrare au fost extrase din checker-ul pus la dispozitie pentru testarea temei. Algoritmul a fost rulat pe un sistem cu procesor **AMD Ryzen 9 7900X 12-Core, 32GB RAM**, sistem de operare nativ **Windows 11** dar rulat pe o masina virtuala **WSL2**, cu sistem de operare **Ubuntu 24.04.3 LTS**, versiune **JDK 25.0.1**. Fiecare configuratie (articole + numar de thread-uri) a fost rulata de **5** ori iar datele din tabele reprezinta timpul mediu masurat in secunde.

| Articles | 1 Thread | 2 Threads | 3 Threads | 4 Threads | 5 Threads | 6 Threads |
|----------|----------|-----------|-----------|-----------|-----------|-----------|
| 17 000   | 2.661    | 1.810     | 1.413     | 1.393     | 0.896     | 1.348     |
| 36 000   | 4.663    | 1.863     | 2.340     | 2.046     | 2.010     | 2.034     |
| 53 000   | 7.809    | 3.830     | 2.123     | 1.873     | 2.593     | 2.482     |
| 70 000   | 8.751    | 4.963     | 2.982     | 2.476     | 3.170     | 3.151     |
| 90 000   | 10.10    | 4.903     | 3.566     | 4.223     | 3.716     | 3.643     |

### 3 Analiza complexității de timp

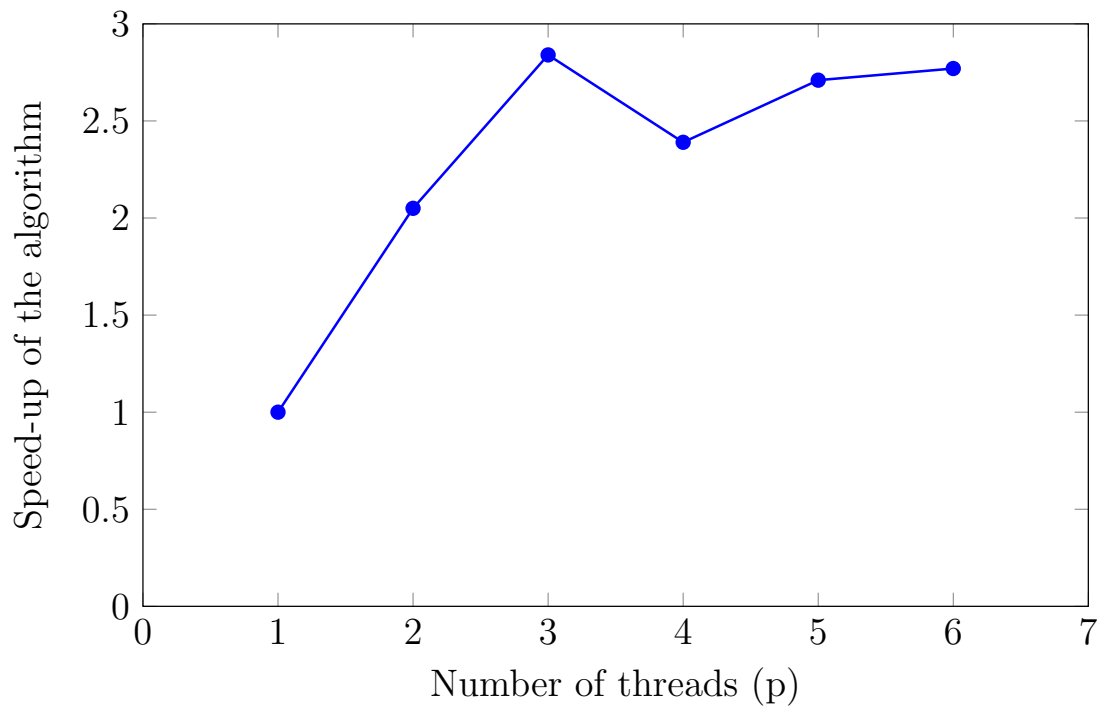


Putem observa ca pentru date de intrare de dimensiuni de pana la  $10^5$  algoritmul are o complexitate de timp aproximativ liniara atunci cand ruleaza cu un singur thread de tip "worker". Se observa ca aceasta analiza devine mai complicata atunci cand se alocă mai multe thread-uri de tip "worker" - un set de date de intrare mai mare ar fi mai decisiv in acest caz. De asemenea, desi complexitatea teoretica a algoritmului este  $O(n \cdot \log(n))$ , datele de intrare nu sunt suficient de mari pentru a reflecta aceasta complexitate asimptotica.

### 4 Analiza gradului de paralelizare

In continuare vom determina speed-up-ul ( $S(p) = \frac{T(1)}{T(p)}$ ), respectiv eficienta ( $E(p) = \frac{S(p)}{p}$ ) algoritmului pentru a estima gradul de paralelizare. Vom studia acesti coeficienti pentru cel mai mare set de date de intrare analizat (90000 de articole).

| Thread(s) | T(1)  | T(p)  | S(p)  | E(p) |
|-----------|-------|-------|-------|------|
| 1         | 10.10 | 10.10 | 1.00x | 1.00 |
| 2         | 10.10 | 4.903 | 2.05x | 1.02 |
| 3         | 10.10 | 3.556 | 2.84x | 0.94 |
| 4         | 10.10 | 4.223 | 2.39x | 0.59 |
| 5         | 10.10 | 3.716 | 2.71x | 0.54 |
| 6         | 10.10 | 3.643 | 2.77x | 0.46 |



Observam ca performanta creste pana la  $p = 3$ , urmand ca aceasta sa scada pentru  $p = 5$  (?) si sa se stabilizeze pentru  $p \geq 5$ . In urma analizei performantei folosind un profiler, instabilitatea pentru  $p = 4$  se datoreaza imprevizibilitatii JVM privind procesul de "garbage collecting". De asemenea, ruland aceste teste pe un laptop, timpii de executie variaua *ocasional* cu  $\pm 25\%$ !

Folosind un profiler, am determinat ca citirea input-urilor din linia de comanda si afisarea rezultatelor finale ocupa  $\approx 10\%$  din timpul total de executie, iar sincronizarea rezultatelor generate de thread-urile de tip "worker"  $\approx 12\%$ .

De asemenea, folosind aceleasi unelte se observa ca timpul necesar citirii metadatelor din fisierele de tip JSON nu scade liniar cu numarul de thread-uri folosite - intrucat sincronizarea este non-blocking, cel mai probabil bottleneck-ul acestei etape reprezinta viteza de citire a disk-ului pe care sunt stocate metadatele.

Etapa de indexare/procesare a articolelor, desi nu scade nici ea liniar cu numarul de thread-uri, are o curba asociata timpului de executie in raport cu numarul de thread-uri mai apropiata de o functie liniara. Un bottleneck la acest stadiu il reprezinta cel mai probabil structurile de date in care sunt stocate rezultatele fiecarui thread "worker". O posibila strategie de optimizare ar fi pre-alocarea memoriei necesare fiecarai structuri, insa acest lucru necesita o analiza mai detaliata pentru a se determina care ar fi dimensiunea optima a acestei pre-alocări.